JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Interface ChronoZonedDateTime<D extends ChronoLocalDate>

Type Parameters:

`D - the concrete type for the date of this date-time`

All Superinterfaces:

`Comparable<ChronoZonedDateTime<?>>, Temporal, TemporalAccessor`

All Known Implementing Classes:

`ZonedDateTime`

```
public interface ChronoZonedDateTime<D extends ChronoLocalDate>
extends Temporal, Comparable<ChronoZonedDateTime<?>>
```

A date-time with a time-zone in an arbitrary chronology, intended for advanced globalization use cases.

Most applications should declare method signatures, fields and variables as `ZonedDateTime`, not this interface.

A `ChronoZonedDateTime` is the abstract representation of an offset date-time where the `Chronology chronology`, or calendar system, is pluggable. The date-time is defined in terms of fields expressed by `TemporalField`, where most common implementations are defined in `ChronoField`. The chronology defines how the calendar system operates and the meaning of the standard fields.

When to use this interface

The design of the API encourages the use of `ZonedDateTime` rather than this interface, even in the case where the application needs to deal with multiple calendar systems. The rationale for this is explored in detail in `ChronoLocalDate`.

Ensure that the discussion in ChronoLocalDate has been read and understood before using this interface.

Implementation Requirements:

This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.

Since:

1.8

Method Summary

Default M

Modifier and Type	Method	De
default int	compareTo(ChronoZonedDateTime<?> other)	C ti ti cl
boolean	equals(Object obj)	C ti ar
default String	format(DateTimeFormatter formatter)	F u: fo
static ChronoZonedDateTime<?>	from(TemporalAccessor temporal)	O Cl fr ol
default int	get(TemporalField field)	G sp i
default Chronology	getChronology()	G th
default long	getLong(TemporalField field)	G sp l
ZoneOffset	getOffset()	G su

ZoneId	getZone()	G as
int	hashCode()	A da
default boolean	isAfter(ChronoZonedDateTime<?> other)	C th th da
default boolean	isBefore(ChronoZonedDateTime<?> other)	C th th da
default boolean	isEqual(ChronoZonedDateTime<?> other)	C th to da
boolean	isSupported(TemporalField field)	C fi
default boolean	isSupported(TemporalUnit unit)	C u
default ChronoZonedDateTime<D>	minus(long amountToSubtract, TemporalUnit unit)	R th ol sp su
default ChronoZonedDateTime<D>	minus(TemporalAmount amount)	R th ol su
ChronoZonedDateTime<D>	plus(long amountToAdd, TemporalUnit unit)	R th ol sp
default ChronoZonedDateTime<D>	plus(TemporalAmount amount)	R th ol a
default <R> R	query(TemporalQuery<R> query)	

		Q u q
default **ValueRange**	**range**(**TemporalField** field)	G va fi
static **Comparator**<**ChronoZonedDateTime**<?>>	**timeLineOrder**()	G co Cl in ig cl
default long	**toEpochSecond**()	C to se ep 1
default **Instant**	**toInstant**()	C to
default **D**	**toLocalDate**()	G pa
ChronoLocalDateTime<**D**>	**toLocalDateTime**()	G ti ti
default **LocalTime**	**toLocalTime**()	G pa
String	**toString**()	O as
default **ChronoZonedDateTime**<**D**>	**with**(**TemporalAdjuster** adjuster)	R ol as ad
ChronoZonedDateTime<**D**>	**with**(**TemporalField** field, long newValue)	R th ol sp
ChronoZonedDateTime<**D**>	**withEarlierOffsetAtOverlap**()	R da zo

		e o li
ChronoZonedDateTime<D>	withLaterOffsetAtOverlap()	R d z o a o
ChronoZonedDateTime<D>	withZoneSameInstant(ZoneId zone)	R d d re
ChronoZonedDateTime<D>	withZoneSameLocal(ZoneId zone)	R d d re d

Methods inherited from interface java.time.temporal.Temporal

until

Method Detail

timeLineOrder

```
static Comparator<ChronoZonedDateTime<?>> timeLineOrder()
```

Gets a comparator that compares `ChronoZonedDateTime` in time-line order ignoring the chronology.

This comparator differs from the comparison in `compareTo(java.time.chrono.ChronoZonedDateTime<?>)` in that it only compares the underlying instant and not the chronology. This allows dates in different calendar systems to be compared based on the position of the date-time on the instant time-line. The underlying comparison is equivalent to comparing the epoch-second and nano-of-second.

Returns:

a comparator that compares in time-line order ignoring the chronology

See Also:

isAfter(java.time.chrono.ChronoZonedDateTime<?>),
isBefore(java.time.chrono.ChronoZonedDateTime<?>),
isEqual(java.time.chrono.ChronoZonedDateTime<?>)

from

`static ChronoZonedDateTime<?> from(TemporalAccessor temporal)`

Obtains an instance of `ChronoZonedDateTime` from a temporal object.

This creates a zoned date-time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoZonedDateTime`.

The conversion extracts and combines the chronology, date, time and zone from the temporal object. The behavior is equivalent to using `Chronology.zonedDateTime(TemporalAccessor)` with the extracted chronology. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ChronoZonedDateTime::from`.

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the date-time, not null`

Throws:

`DateTimeException - if unable to convert to a ChronoZonedDateTime`

See Also:

`Chronology.zonedDateTime(TemporalAccessor)`

range

`default ValueRange range(TemporalField field)`

Description copied from interface: `TemporalAccessor`
Gets the range of valid values for the specified field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value. The value of this temporal object is used to enhance the accuracy of the returned range. If the date-time cannot return the range, because the field is unsupported or for some other reason, an exception will be thrown.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field - the field to query the range for, not null`

Returns:

`the range of valid values for the field, not null`

get

```
default int get(TemporalField field)
```

Description copied from interface: `TemporalAccessor`

Gets the value of the specified field as an `int`.

This queries the date-time for the value of the specified field. The returned value will always be within the valid range of values for the field. If the date-time cannot return the value, because the field is unsupported or for some other reason, an exception will be thrown.

Specified by:

`get` in interface `TemporalAccessor`

Parameters:

`field - the field to get, not null`

Returns:

`the value for the field, within the valid range of values`

getLong

```
default long getLong(TemporalField field)
```

Description copied from interface: `TemporalAccessor`

Gets the value of the specified field as a `long`.

This queries the date-time for the value of the specified field. The returned value may be outside the valid range of values for the field. If the date-time cannot return the value, because the field is unsupported or for some other reason, an exception will be thrown.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

field - the field to get, not null

Returns:

the value for the field

toLocalDate

```
default D toLocalDate()
```

Gets the local date part of this date-time.

This returns a local date with the same year, month and day as this date-time.

Returns:

the date part of this date-time, not null

toLocalTime

```
default LocalTime toLocalTime()
```

Gets the local time part of this date-time.

This returns a local time with the same hour, minute, second and nanosecond as this date-time.

Returns:

the time part of this date-time, not null

toLocalDateTime

```
ChronoLocalDateTime<D> toLocalDateTime()
```

Gets the local date-time part of this date-time.

This returns a local date with the same year, month and day as this date-time.

Returns:

the local date-time part of this date-time, not null

getChronology

```
default Chronology getChronology()
```

Gets the chronology of this date-time.

The `Chronology` represents the calendar system in use. The era and other fields in `ChronoField` are defined by the chronology.

Returns:

`the chronology, not null`

getOffset

`ZoneOffset getOffset()`

Gets the zone offset, such as '+01:00'.

This is the offset of the local date-time from UTC/Greenwich.

Returns:

`the zone offset, not null`

getZone

`ZoneId getZone()`

Gets the zone ID, such as 'Europe/Paris'.

This returns the stored time-zone id used to determine the time-zone rules.

Returns:

`the zone ID, not null`

withEarlierOffsetAtOverlap

`ChronoZonedDateTime<D> withEarlierOffsetAtOverlap()`

Returns a copy of this date-time changing the zone offset to the earlier of the two valid offsets at a local time-line overlap.

This method only has any effect when the local time-line overlaps, such as at an autumn daylight savings cutover. In this scenario, there are two valid offsets for the local date-time. Calling this method will return a zoned date-time with the earlier of the two selected.

If this method is called when it is not an overlap, `this` is returned.

This instance is immutable and unaffected by this method call.

Returns:

`a ChronoZonedDateTime based on this date-time with the earlier offset, not null`

Throws:

`DateTimeException` - if no rules can be found for the zone

`DateTimeException` - if no rules are valid for this date-time

withLaterOffsetAtOverlap

`ChronoZonedDateTime<D> withLaterOffsetAtOverlap()`

Returns a copy of this date-time changing the zone offset to the later of the two valid offsets at a local time-line overlap.

This method only has any effect when the local time-line overlaps, such as at an autumn daylight savings cutover. In this scenario, there are two valid offsets for the local date-time. Calling this method will return a zoned date-time with the later of the two selected.

If this method is called when it is not an overlap, `this` is returned.

This instance is immutable and unaffected by this method call.

Returns:

a ChronoZonedDateTime based on this date-time with the later offset, not null

Throws:

`DateTimeException` - if no rules can be found for the zone

`DateTimeException` - if no rules are valid for this date-time

withZoneSameLocal

`ChronoZonedDateTime<D> withZoneSameLocal(ZoneId zone)`

Returns a copy of this date-time with a different time-zone, retaining the local date-time if possible.

This method changes the time-zone and retains the local date-time. The local date-time is only changed if it is invalid for the new zone.

To change the zone and adjust the local date-time, use `withZoneSameInstant(ZoneId)`.

This instance is immutable and unaffected by this method call.

Parameters:

zone - the time-zone to change to, not null

Returns:

a ChronoZonedDateTime based on this date-time with the requested zone, not null

withZoneSameInstant

`ChronoZonedDateTime<D> withZoneSameInstant(ZoneId zone)`

Returns a copy of this date-time with a different time-zone, retaining the instant.

This method changes the time-zone and retains the instant. This normally results in a change to the local date-time.

This method is based on retaining the same instant, thus gaps and overlaps in the local time-line have no effect on the result.

To change the offset while keeping the local time, use `withZoneSameLocal(ZoneId)`.

Parameters:

`zone` - the time-zone to change to, not null

Returns:

a ChronoZonedDateTime based on this date-time with the requested zone, not null

Throws:

`DateTimeException` - if the result exceeds the supported date range

isSupported

`boolean isSupported(TemporalField field)`

Checks if the specified field is supported.

This checks if the specified field can be queried on this date-time. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

The set of supported fields is defined by the chronology and normally includes all `ChronoField` fields.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field can be queried, false if not`

isSupported

```
default boolean isSupported(TemporalUnit unit)
```

Checks if the specified unit is supported.

This checks if the specified unit can be added to or subtracted from this date-time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

The set of supported units is defined by the chronology and normally includes all `ChronoUnit` units except `FOREVER`.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported in interface Temporal`

Parameters:

`unit - the unit to check, null returns false`

Returns:

`true if the unit can be added/subtracted, false if not`

with

```
default ChronoZonedDateTime<D> with(TemporalAdjuster adjuster)
```

Returns an adjusted object of the same type as this object with the adjustment made.

This adjusts this date-time according to the rules of the specified adjuster. A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in `TemporalAdjusters`. These include finding the "last day of the month" and "next Wednesday". The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

Some example code indicating how and why this method is used:

```
  date = date.with(Month.JULY);        // most key classes implement TemporalAdju
  date = date.with(lastDayOfMonth());  // static import from Adjusters
```

```
  date = date.with(next(WEDNESDAY));   // static import from Adjusters and DayOfW
```

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

with

```
ChronoZonedDateTime<D> with(TemporalField field,
                            long newValue)
```

Returns an object of the same type as this object with the specified field altered.

This returns a new object based on this one with the value for the specified field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`with` in interface `Temporal`

Parameters:

`field` - the field to set in the result, not null

`newValue` - the new value of the field in the result

Returns:

an object of the same type with the specified field set, not null

Throws:

`DateTimeException` - if the field cannot be set

`ArithmeticException` - if numeric overflow occurs

plus

```
default ChronoZonedDateTime<D> plus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount added.

This adjusts this temporal, adding according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.plus(period);                // add a Period instance
  date = date.plus(duration);              // add a Duration instance
  date = date.plus(workingDays(6));        // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amount` - the amount to add, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
ChronoZonedDateTime<D> plus(long amountToAdd,
                            TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period added.

This method returns a new object based on this one with the specified period added. For example, on a `LocalDate`, this could be used to add a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then adding one month would be unclear. In cases like this, the field is responsible for resolving the result.

Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd` - the amount of the specified unit to add, may be negative

`unit` - the unit of the amount to add, not null

Returns:

an object of the same type with the specified period added, not null

Throws:

`DateTimeException` - if the unit cannot be added

`ArithmeticException` - if numeric overflow occurs

minus

`default ChronoZonedDateTime<D> minus(TemporalAmount amount)`

Returns an object of the same type as this object with an amount subtracted.

This adjusts this temporal, subtracting according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.minus(period);               // subtract a Period instance
  date = date.minus(duration);             // subtract a Duration instance
  date = date.minus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amount` - the amount to subtract, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

ArithmeticException - if numeric overflow occurs

minus

```
default ChronoZonedDateTime<D> minus(long amountToSubtract,
                                     TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period subtracted.

This method returns a new object based on this one with the specified period subtracted. For example, on a `LocalDate`, this could be used to subtract a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st March, then subtracting one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

minus in interface Temporal

Parameters:

amountToSubtract - the amount of the specified unit to subtract, may be negative

unit - the unit of the amount to subtract, not null

Returns:

an object of the same type with the specified period subtracted, not null

Throws:

DateTimeException - if the unit cannot be subtracted

ArithmeticException - if numeric overflow occurs

query

```
default <R> R query(TemporalQuery<R> query)
```

Queries this date-time using the specified query.

This queries this date-time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

format

```
default String format(DateTimeFormatter formatter)
```

Formats this date-time using the specified formatter.

This date-time will be passed to the formatter to produce a string.

The default implementation must behave as follows:

```
  return formatter.format(this);
```

Parameters:

`formatter` - the formatter to use, not null

Returns:

the formatted date-time string, not null

Throws:

`DateTimeException` - if an error occurs during printing

toInstant

```
default Instant toInstant()
```

Converts this date-time to an `Instant`.

This returns an `Instant` representing the same point on the time-line as this date-time. The calculation combines the local date-time and offset.

Returns:

`an Instant representing the same instant, not null`

toEpochSecond

```
default long toEpochSecond()
```

Converts this date-time to the number of seconds from the epoch of 1970-01-01T00:00:00Z.

This uses the local date-time and offset to calculate the epoch-second value, which is the number of elapsed seconds from 1970-01-01T00:00:00Z. Instants on the time-line after the epoch are positive, earlier are negative.

Returns:

`the number of seconds from the epoch of 1970-01-01T00:00:00Z`

compareTo

```
default int compareTo(ChronoZonedDateTime<?> other)
```

Compares this date-time to another date-time, including the chronology.

The comparison is based first on the instant, then on the local date-time, then on the zone ID, then on the chronology. It is "consistent with equals", as defined by `Comparable`.

If all the date-time objects being compared are in the same chronology, then the additional chronology stage is not required.

This default implementation performs the comparison defined above.

Specified by:

`compareTo in interface Comparable<ChronoZonedDateTime<?>>`

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`the comparator value, negative if less, positive if greater`

isBefore

```
default boolean isBefore(ChronoZonedDateTime<?> other)
```

Checks if the instant of this date-time is before that of the specified date-time.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoZonedDateTime<?>)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().isBefore(dateTime2.toInstant());`.

This default implementation performs the comparison based on the epoch-second and nano-of-second.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if this point is before the specified date-time`

isAfter

```
default boolean isAfter(ChronoZonedDateTime<?> other)
```

Checks if the instant of this date-time is after that of the specified date-time.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoZonedDateTime<?>)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().isAfter(dateTime2.toInstant());`.

This default implementation performs the comparison based on the epoch-second and nano-of-second.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if this is after the specified date-time`

isEqual

```
default boolean isEqual(ChronoZonedDateTime<?> other)
```

Checks if the instant of this date-time is equal to that of the specified date-time.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoZonedDateTime<?>)` and `equals(java.lang.Object)` in that it only compares the instant of the date-time. This is equivalent to using `dateTime1.toInstant().equals(dateTime2.toInstant());`.

This default implementation performs the comparison based on the epoch-second and nano-of-second.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if the instant equals the instant of the specified date-time`

equals

`boolean equals(Object obj)`

Checks if this date-time is equal to another date-time.

The comparison is based on the offset date-time and the zone. To compare for the same instant on the time-line, use `compareTo(java.time.chrono.ChronoZonedDateTime<?>)`. Only objects of type `ChronoZonedDateTime` are compared, other types return false.

Overrides:

`equals in class Object`

Parameters:

`obj - the object to check, null returns false`

Returns:

`true if this is equal to the other date-time`

See Also:

`Object.hashCode(), HashMap`

hashCode

`int hashCode()`

A hash code for this date-time.

Overrides:

`hashCode in class Object`

Returns:

`a suitable hash code`

See Also:

`Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)`

toString

`String toString()`

Outputs this date-time as a `String`.

The output will include the full zoned date-time.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this date-time, not null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.